Machine Learning II

Miguel García Moreno

2026-04-20

Contents

1 Feature selection and boosting **3**

1.1 Assignment submission 3

1.2 Data description 3

1.3 Tasks 3

```
# Package installation
paquetes<-c('ggplot2','cluster','mlbench','FSelector','ROCR','MASS','mlbench','caret','xgboost','GA')
for (i in 1:length(paquetes))
{
  if(require(paquetes[i],character.only = TRUE)) cat(paquetes[i], " loaded successfully\n")
  else {
    cat("Trying to install",paquetes[i])
    install.packages(paquetes[i],dependencies = TRUE)
    if(require(paquetes[i],character.only = TRUE)) cat(paquetes[i], " loaded successfully")
    else {
      stop("Could not install ", paquetes[i])
    }
  }
}
```

```
## ggplot2  loaded successfully

## cluster  loaded successfully

## mlbench  loaded successfully

## FSelector  loaded successfully

## ROCR  loaded successfully

## MASS  loaded successfully
## mlbench  loaded successfully

## caret  loaded successfully

## xgboost  loaded successfully

## GA  loaded successfully
```

1 Feature selection and boosting

1.1 Assignment submission

All R instruction files and presentations of the results are to be submitted in a `.zip` archive via the course messaging tool to **Rafael Pino** (`.RMarkdown` files are accepted).

Deadline: 10 May.

1.2 Data description

We use the same *data frame* `datos` generated in the *Machine Learning I* assignment.

Recall that:

- It contains **1000 cases**.
- It includes the variable `label` (categorical, distinguishing two garment types).
- It contains **10 potentially predictive variables** for `label`.
- These variables correspond to the content of 10 pixels selected among the 784 greyscale pixels of each 28×28 image.

1.3 Tasks

1. **Data partition**

    - Split the data into *train* and *test* subsets (75% and 25%). We replicate the partition used in the *Machine Learning I* assignment to ensure comparability of results.

```
# First five digits of the national ID (seed)
m <- 29509
# Load data-generation script
source("GenerarDatos.R")

set.seed(12345)
m <- dim(datos)[1] # number of cases
test <- sample(1:m, size = 0.25 * m, replace = FALSE)
# training set
x_train <- datos[-test,]
# test set
x_test <- datos[test,]
```

2. **Feature selection and LDA**

    - Apply **two feature-selection techniques**. We start with attribute-importance-based selection.

```
pesos_1 = FSelector::cfs(label~.,datos)
pesos_1
```

```
## [1] "pixel125"
```

We also try genetic-algorithm-based feature selection.

```
set.seed(12345)
ga_control = gafsControl(functions = rfGA, method = "cv", number = 4)
pesos_2 = gafs(x = datos[, !(names(datos) %in% "label")], y = datos$label,
                  popSize=20, pcrossover=0.8, pmutation=0.1, iters = 3,
                  gafsControl = ga_control)
print(pesos_2)
```

```
##
## Genetic Algorithm Feature Selection
##
## 1000 samples
## 10 predictors
## 2 classes: 'Pullover', 'Ankle boot'
##
## Maximum generations: 3
## Population per generation: 20
## Crossover probability: 0.8
## Mutation probability: 0.1
## Elitism: 0
##
## Internal performance values: Accuracy, Kappa
## Subset selection driven to maximize internal Accuracy
##
## External performance values: Accuracy, Kappa
## Best iteration chose by maximizing external Accuracy
## External resampling method: Cross-Validated (4 fold)
##
## During resampling:
##   * the top 5 selected variables (out of a possible 10):
##     pixel125 (100%), pixel528 (100%), pixel733 (100%), pixel745 (100%), pixel81 (100%)
##   * on average, 8 variables were selected (min = 7, max = 10)
##
## In the final search using the entire training set:
##    * 8 features selected at iteration 1 including:
##      pixel81, pixel125, pixel161, pixel211, pixel640 ...
##    * external performance at this iteration is
##
##    Accuracy       Kappa
##      0.9620      0.9238
```

We select the five most relevant variables highlighted by the function output, while noting that it actually selects eight ("In the final search using the entire training set: * 8 features selected at iteration 1 including: pixel81, pixel125, pixel161, pixel211, pixel640 ..")

- With each selection, build a classification model based on **Fisher's linear discriminant analysis (LDA)**.

First selection of variables:

```
modeloADL_1 = lda(label ~ pixel125+pixel528+pixel733+pixel745, datos)
modeloADL_1
```

```
## Call:
## lda(label ~ pixel125 + pixel528 + pixel733 + pixel745, data = datos)
##
## Prior probabilities of groups:
##   Pullover Ankle boot
##      0.523      0.477
##
## Group means:
##             pixel125 pixel528  pixel733  pixel745
## Pullover   148.01618 136.6990 100.59319 64.287957
## Ankle boot  14.29231 189.1095  18.99865  8.160229
##
## Coefficients of linear discriminants:
##                    LD1
## pixel125 -0.014629900
## pixel528  0.011556774
## pixel733 -0.005678291
## pixel745 -0.002634348
```

The discriminant direction formula is

$$LD1 = -0.014629900\, x_{125} + 0.011556774\, x_{528} - 0.005678291\, x_{733} - 0.002634348\, x_{745}$$

If the separating hyperplane equation is needed, it suffices to consider

$$LD1 + c = 0$$

where c is the cut-off point.

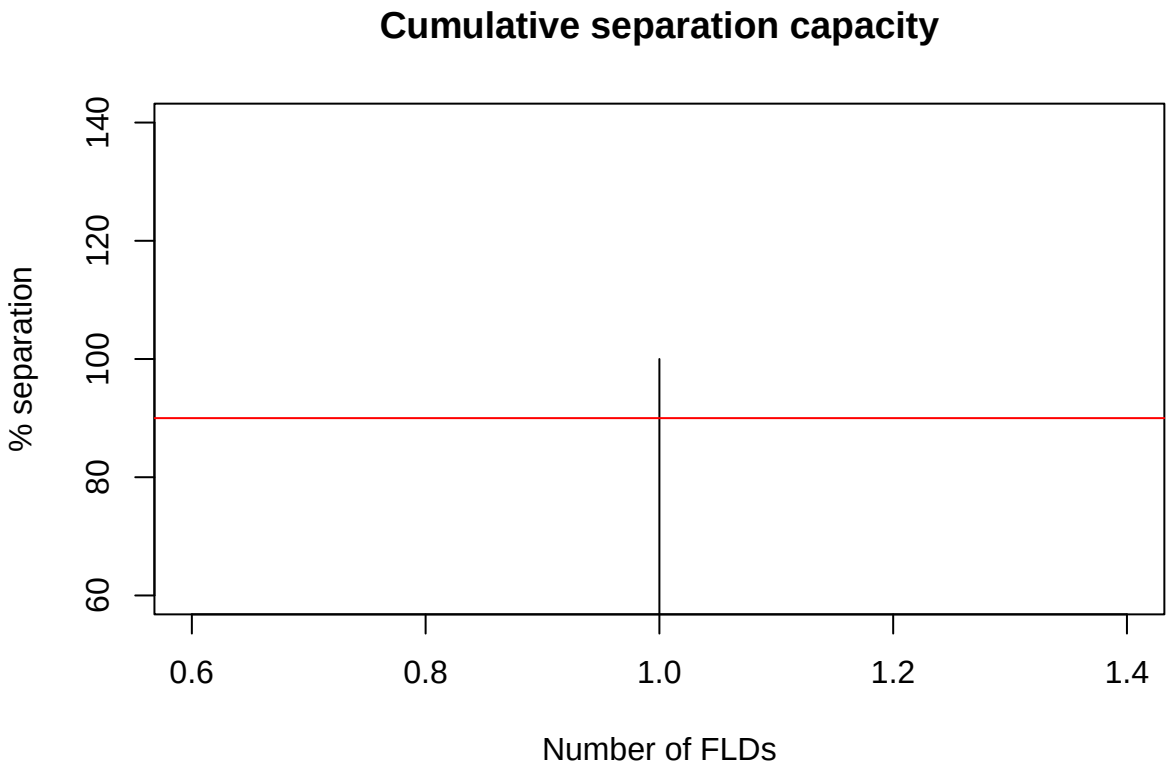
Second selection of variables:

```
modeloADL_2 = lda(label ~ pixel125, datos)
modeloADL_2
```

```
## Call:
## lda(label ~ pixel125, data = datos)
##
## Prior probabilities of groups:
##   Pullover Ankle boot
##      0.523      0.477
##
## Group means:
##             pixel125
## Pullover   148.01618
## Ankle boot  14.29231
##
## Coefficients of linear discriminants:
##                  LD1
## pixel125 0.01701901
```

- Evaluate and compare both models.

```r
plot(cumsum(100*modeloADL_1$svd^2/sum(modeloADL_1$svd^2) ),type="h",
     main="Cumulative separation capacity",
     xlab="Number of FLDs",
     ylab="% separation")
abline(h=90,col="red")
```



With a binary response we obtain a single separation, so this plot is not informative. We therefore study the confusion matrix and associated metrics.

```r
# ==========================
# MODEL 1
# ==========================
pred1 <- predict(modeloADL_1, newdata = x_test)$class

cm1 <- confusionMatrix(
  factor(pred1),
  factor(x_test$label),
  positive = "Ankle boot"
)

cm1
```

```
## Confusion Matrix and Statistics
##
##             Reference
## Prediction   Pullover Ankle boot
```

```
##   Pullover        119          5
##   Ankle boot        5        121
##
##                Accuracy : 0.96
##                  95% CI : (0.9277, 0.9807)
##     No Information Rate : 0.504
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.92
##
##  Mcnemar's Test P-Value : 1
##
##             Sensitivity : 0.9603
##             Specificity : 0.9597
##          Pos Pred Value : 0.9603
##          Neg Pred Value : 0.9597
##              Prevalence : 0.5040
##          Detection Rate : 0.4840
##    Detection Prevalence : 0.5040
##       Balanced Accuracy : 0.9600
##
##        'Positive' Class : Ankle boot
##
```

```r
# Extract metrics
metrics1 <- c(
  Accuracy  = cm1$overall["Accuracy"],
  Precision = cm1$byClass["Pos Pred Value"],
  Recall    = cm1$byClass["Sensitivity"],
  F1_Score  = cm1$byClass["F1"]
)

metrics1
```

```
##        Accuracy.Accuracy Precision.Pos Pred Value       Recall.Sensitivity
##                0.9600000                0.9603175                0.9603175
##              F1_Score.F1
##                0.9603175
```

For the first model we obtain about 90% accuracy, together with a 95% confidence interval for that accuracy. The no-information rate provides a baseline; the line P-Value [Acc > NIR] : < 2.2e-16 tests whether the model beats that baseline. The tiny p-value lets us reject the null and conclude that the model performs better.

```r
# ========================
# MODEL 2
# ========================
pred2 <- predict(modeloADL_2, newdata = x_test)$class

cm2 <- confusionMatrix(
  factor(pred2),
  factor(x_test$label),
  positive = "Ankle boot"
```

```
)

cm2
```

```
## Confusion Matrix and Statistics
##
##             Reference
## Prediction   Pullover Ankle boot
##   Pullover         98          4
##   Ankle boot       26        122
##
##                Accuracy : 0.88
##                  95% CI : (0.8331, 0.9176)
##     No Information Rate : 0.504
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.7596
##
##  Mcnemar's Test P-Value : 0.000126
##
##             Sensitivity : 0.9683
##             Specificity : 0.7903
##          Pos Pred Value : 0.8243
##          Neg Pred Value : 0.9608
##              Prevalence : 0.5040
##          Detection Rate : 0.4880
##    Detection Prevalence : 0.5920
##       Balanced Accuracy : 0.8793
##
##        'Positive' Class : Ankle boot
##
```

```
# Extract metrics
metrics2 <- c(
  Accuracy  = cm2$overall["Accuracy"],
  Precision = cm2$byClass["Pos Pred Value"],
  Recall    = cm2$byClass["Sensitivity"],
  F1_Score  = cm2$byClass["F1"]
)

metrics2
```

```
##        Accuracy.Accuracy Precision.Pos Pred Value       Recall.Sensitivity
##                0.8800000                0.8243243                0.9682540
##              F1_Score.F1
##                0.8905109
```

```
# ========================
# COMPARISON TABLE
# ========================
comparison <- rbind(
  Modelo_ADL_1 = metrics1,
```

```
  Modelo_ADL_2 = metrics2
)

comparison
```

```
##              Accuracy.Accuracy Precision.Pos Pred Value Recall.Sensitivity
## Modelo_ADL_1              0.96                0.9603175          0.9603175
## Modelo_ADL_2              0.88                0.8243243          0.9682540
##              F1_Score.F1
## Modelo_ADL_1   0.9603175
## Modelo_ADL_2   0.8905109
```

Both models are strong (accuracy above 0.88). The first is clearly better, but also more complex because it uses five predictors. The key difference is the ability to predict the "Pullover" class: the second model, with a single predictor, misclassifies 26 cases, while the first does much better. This shows up in specificity and, of course, in precision.

3. **Gradient Boosting model**

   - Using the `train` function from the `caret` package, build and evaluate a **Gradient Boosting** model.

```
train_control <- trainControl(method = "cv", number = 10)

grid <- expand.grid(n.trees = c(100,250,500),
                    interaction.depth = c(1,3,5,7),
                    shrinkage = c(0.01, 0.1),
                    n.minobsinnode = 25)

tuned_modelo <- train(label ~ .,
                      data = x_train,
                      method = "gbm", verbose=FALSE,
                      distribution="bernoulli",
                      trControl = train_control,
                      tuneGrid = grid)
tuned_modelo
```

```
## Stochastic Gradient Boosting
##
## 750 samples
##  10 predictor
##   2 classes: 'Pullover', 'Ankle boot'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 675, 675, 675, 674, 675, 675, ...
## Resampling results across tuning parameters:
##
##   shrinkage  interaction.depth  n.trees  Accuracy   Kappa
##   0.01       1                  100      0.9346287  0.8682391
##   0.01       1                  250      0.9346287  0.8682391
##   0.01       1                  500      0.9479445  0.8952846
```

```
##   0.01       3                  100      0.9359621  0.8708934
##   0.01       3                  250      0.9426292  0.8843752
##   0.01       3                  500      0.9640166  0.9275849
##   0.01       5                  100      0.9359621  0.8708934
##   0.01       5                  250      0.9439625  0.8871266
##   0.01       5                  500      0.9653499  0.9302490
##   0.01       7                  100      0.9359621  0.8708934
##   0.01       7                  250      0.9466292  0.8925315
##   0.01       7                  500      0.9666657  0.9329063
##   0.10       1                  100      0.9559445  0.9114043
##   0.10       1                  250      0.9666833  0.9329996
##   0.10       1                  500      0.9666652  0.9330226
##   0.10       3                  100      0.9706837  0.9409605
##   0.10       3                  250      0.9693504  0.9382772
##   0.10       3                  500      0.9679991  0.9355705
##   0.10       5                  100      0.9653679  0.9303040
##   0.10       5                  250      0.9680166  0.9356286
##   0.10       5                  500      0.9640166  0.9275881
##   0.10       7                  100      0.9666837  0.9329102
##   0.10       7                  250      0.9626833  0.9248660
##   0.10       7                  500      0.9626657  0.9248564
##
## Tuning parameter 'n.minobsinnode' was held constant at a value of 25
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were n.trees = 100, interaction.depth =
##  3, shrinkage = 0.1 and n.minobsinnode = 25.
```

After the grid search the chosen model is n.trees = 250, interaction.depth = 7, shrinkage = 0.1 and n.minobsinnode = 25. Performance metrics:

```r
pred <- predict(tuned_modelo, newdata = x_test)

cm <- confusionMatrix(
  factor(pred),
  factor(x_test$label),
  positive = "Pullover"
)

cm
```

```
## Confusion Matrix and Statistics
##
##             Reference
## Prediction   Pullover Ankle boot
##   Pullover        119          2
##   Ankle boot        5        124
##
##                Accuracy : 0.972
##                  95% CI : (0.9432, 0.9887)
##     No Information Rate : 0.504
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.944
##
```

```
##  Mcnemar's Test P-Value : 0.4497
##
##             Sensitivity : 0.9597
##             Specificity : 0.9841
##          Pos Pred Value : 0.9835
##          Neg Pred Value : 0.9612
##              Prevalence : 0.4960
##          Detection Rate : 0.4760
##    Detection Prevalence : 0.4840
##       Balanced Accuracy : 0.9719
##
##        'Positive' Class : Pullover
##
```

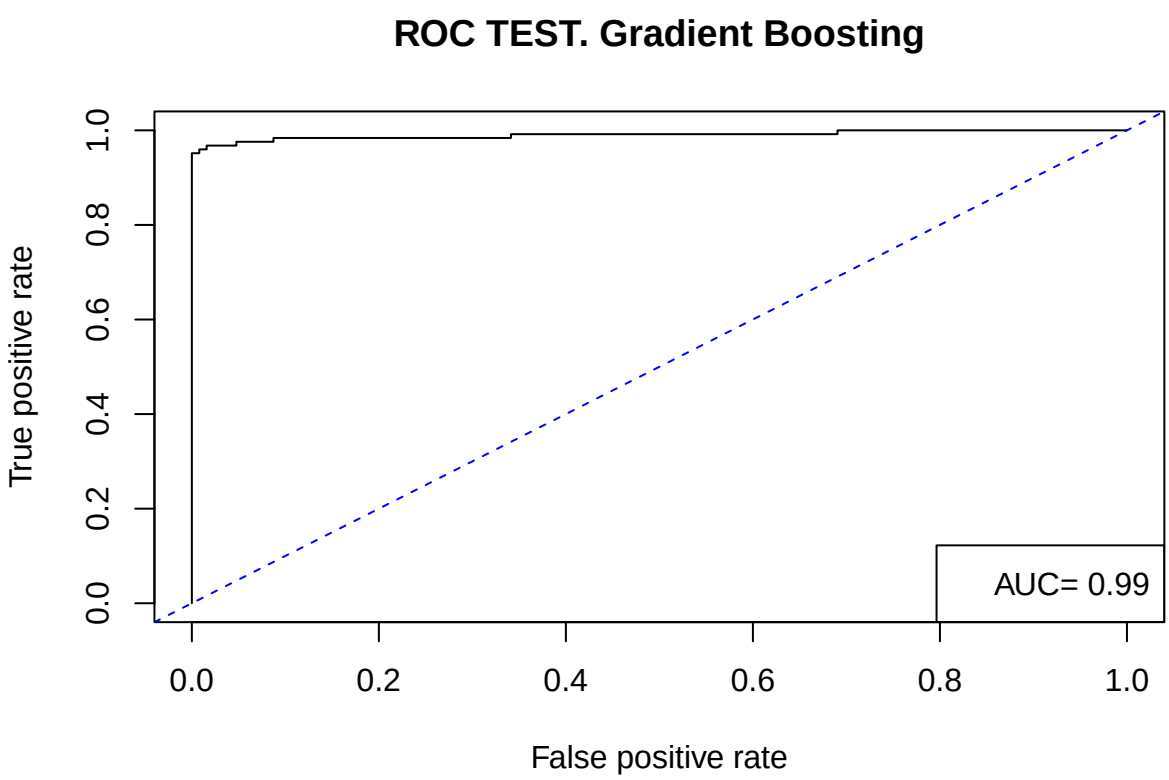
```
# Extract metrics
metrics <- c(
  Accuracy  = cm$overall["Accuracy"],
  Precision = cm$byClass["Pos Pred Value"],
  Recall    = cm$byClass["Sensitivity"],
  F1_Score  = cm$byClass["F1"]
)

metrics
```

```
##        Accuracy.Accuracy Precision.Pos Pred Value       Recall.Sensitivity
##                0.9720000                0.9834711                0.9596774
##              F1_Score.F1
##                0.9714286
```

This model outperforms the previous ones with strong metrics overall. McNemar's Test P-Value rejects the null of equal probability of misclassification in the first and second categories.

```
probabi=predict(tuned_modelo, x_test,type="prob")[,1] #Prob. pos
prediobj<-prediction(probabi,x_test$label)
plot(performance(prediobj, "tpr","fpr"),
     main="ROC TEST. Gradient Boosting",
     xlab="False positive rate",
     ylab="True positive rate")
abline(a=0,b=1,col="blue",lty=2)
auc<- as.numeric(performance(prediobj,"auc")@y.values)
legend("bottomright",legend=paste("AUC=",round(auc,3)))
```



The ROC curve and AUC also confirm that this is a good model.

4. **XGBoost model**
    - Build and evaluate an **XGBoost** model. We prepare the data and fit the model:

```
y_factor <- as.factor(x_train$label)
table(y_factor)
```

```
## y_factor
##   Pullover Ankle boot
##        399        351
```

```
y <- as.integer(y_factor)-1
table(y)
```

```
## y
##   0   1
## 399 351
```

```
X_ent <- data.matrix(x_train[ , -which(names(x_train) == "label")])
dtrain <- xgb.DMatrix(data = X_ent, label = y)

params <- list(
  objective = "binary:logistic",
```

```
  eval_metric = "logloss",
  max_depth = 4,
  learning_rate = 0.1,
  nthread = 2
)
```

```
modelo_xgboost <- xgb.train(
  params = params,
  data = dtrain,
  nrounds = 10
)
```

- Compare the results with the previous model.

```
X_test <- data.matrix(x_test[, -which(names(x_test) == "label")])
dtest <- xgb.DMatrix(data = X_test)
y_test <- as.integer(as.factor(x_test$label))-1


pred_xgboost <- predict(modelo_xgboost, dtest)
pred_class <- ifelse(pred_xgboost > 0.5, 1, 0)
```

```
mconf_xgboost <- table(Pred=pred_class,Real=y_test)
mconf_xgboost
```

```
##     Real
## Pred   0   1
##    0 120   9
##    1   4 117
```

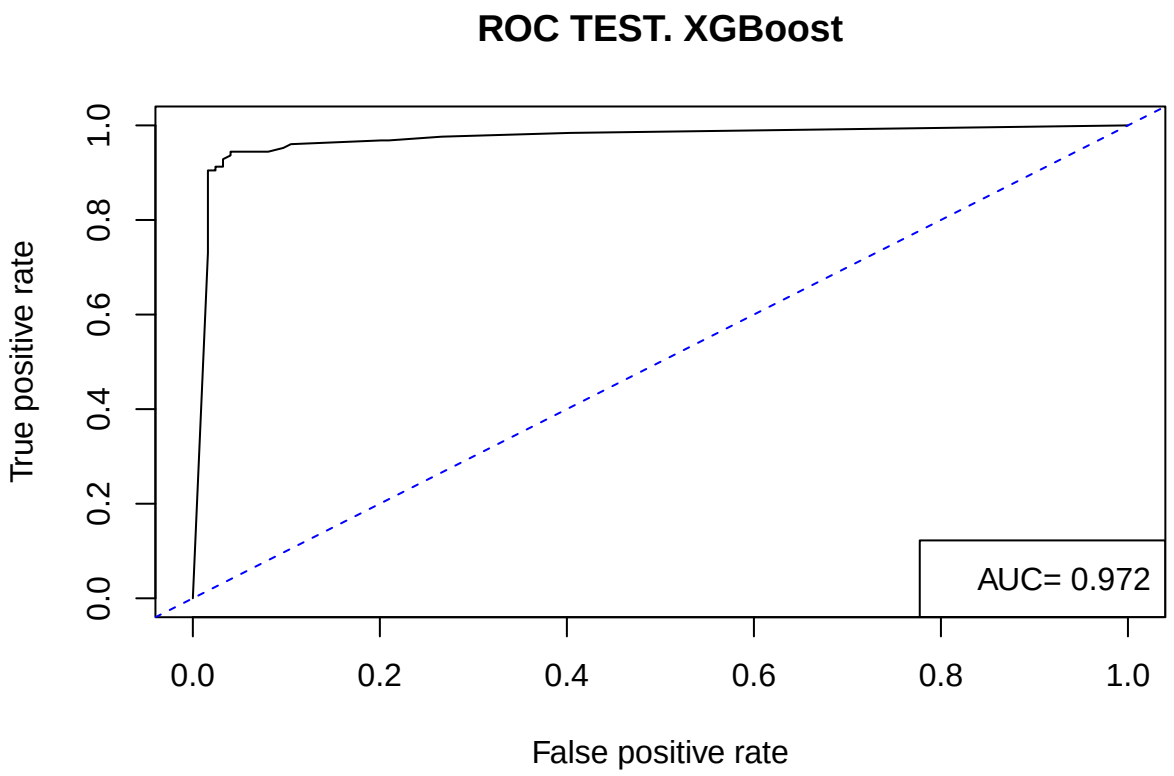
```
Acierto=round(100*sum(diag(prop.table(mconf_xgboost))),2)
SensEspec=round(100*diag(prop.table(mconf_xgboost,1)),2)
cat(" Test accuracy (%)= \t\t",
    Acierto,"%\n",
    "Test sensitivity (pos)= \t\t",
    SensEspec[2],"%\n",
    "Test specificity (neg)=  \t\t",
    SensEspec[1] ,"%\n")
```

```
##  Test accuracy (%)=       94.8 %
##  Test sensitivity (pos)=          96.69 %
##  Test specificity (neg)=          93.02 %
```

```
probabix=predict(modelo_xgboost, dtest)
prediobjx<-prediction(probabix,y_test)
```

Overall we obtain worse results than the previous model on every metric. Preferring the Gradient Boosting model is more sensible.

```r
plot(performance(prediobjx, "tpr","fpr"),
     main="ROC TEST. XGBoost",
     xlab="False positive rate",
     ylab="True positive rate")
abline(a=0,b=1,col="blue",lty=2)
auc<- performance(prediobjx,"auc")@y.values[[1]]
legend("bottomright",legend=paste("AUC=",round(auc,3)))
```



The AUC points to the same conclusion.